OzAgriMate Telemetry Architecture and API

Aditya Singh (s4139812) | ISYS3459 Assignment 4 | 24 October 2026

These sections document the architecture decisions, Telemetry component view, telemetry runtime view and telemetry API contributed by Aditya Singh, plus the repository integration work. The two telemetry endpoints pass 14 automated HTTP tests and a 19-request Postman collection with 59 of 59 assertions passing. Architecture decisions remain Proposed pending team review.

Design basis and scope

The submitted Assignment 2 report names Scalability and Interoperability among its top three driving characteristics: the platform must grow from 5,000 to 50,000 farms with sub-second ingestion and accept LoRaWAN, NB-IoT and other sensor networks without rebuilding core services. ASR22 requires absorbing tenfold seasonal spikes without data loss, ASR3 requires 72-hour edge buffering with resync within 15 minutes, ASR5 requires anomalous readings not to trigger automated actions, ASR15 and ASR16 require adapter-based onboarding and versioned APIs, and C4 limits operations to at most eight engineers. ADR-01 places telemetry in its own Telemetry service, which owns the Telemetry Store and publishes accepted readings through the Event Broker.

This contribution implements reading submission and farm-scoped reading retrieval. The Farm Edge Gateway, Event Broker, autoscaling and the time-series database are target architecture; the prototype uses in-memory partitions and an in-memory outbox.

Contribution coverage

Artefact	Scope and status
ADR-02	Scalability (driving QA A); Proposed
ADR-06	Edge-to-cloud reading contract (technology/interface decision); Proposed
Telemetry component view	Ten components, one-to-one with services/telemetry/ modules
Runtime view 1	Telemetry submission and processing; edge buffering and broker steps marked planned
Telemetry API	POST and GET /farms/{farmId}/telemetry; local implementation on port 8002
Repository integration	run_all.py, requirements.txt, README start-up instructions, clean ZIP builder
Testing evidence	14 telemetry tests; 49 of 49 repository tests; Newman run 19 requests, 59 of 59 assertions

ADR 02 Scalable telemetry ingestion

Field	Architectural decision record
Title	ADR-02 Scale telemetry ingestion with stateless services, farm-partitioned storage and asynchronous accepted-reading events
Status	Proposed
Context	Scalability is the top driving QA: OzAgriMate must grow from 5,000 to 50,000 farms with sub-second ingestion. ASR22 requires absorbing tenfold planting and harvest spikes without data loss; ASR3 requires gateways to resync 72 hours of buffered readings within 15 minutes; C4 limits operations to eight engineers. Alternatives: synchronous fan-out to irrigation, alerts and AI ties ingestion latency to every consumer; one vertically scaled database caps growth; per-farm deployments are unmanageable. Replay bursts must not starve live readings.
Decision	Run Telemetry as stateless, horizontally scaled instances, autoscaled on request rate and broker lag. Acknowledge a reading once stored, then publish a ReadingAccepted event through an outbox to the managed broker, partitioned by farmId, so consumers scale independently. Partition the time-series store by farm and time; queries are farm-scoped and capped at 500 rows. Idempotent reading IDs make replays safe; replay traffic is rate-limited below live traffic. Prototype: partitions, bounded queries, idempotency and outbox only; autoscaling, broker and rate-limiting are not built.
Consequences	Capacity grows by adding instances and partitions, and slow consumers cannot delay acknowledgements. Consumers see eventual consistency and must deduplicate events. Outbox, broker and partition management add cost and monitoring effort. Very active farms can create hot partitions. In-memory prototype results do not demonstrate production throughput.
Compliance	Load-test 50,000 simulated farms at tenfold peak plus 72-hour replay: p95 acknowledgement under one second, zero lost readings, resync within 15 minutes.

Traceability: Scalability (top-three driving QA) and Elasticity; ASR22, ASR3, ASR1; risk R2 (10x load testing); constraints C3 and C4. Refines ADR-01's asynchronous telemetry processing into concrete scaling mechanisms; quarantined readings are never published (ASR5, ADR-03).

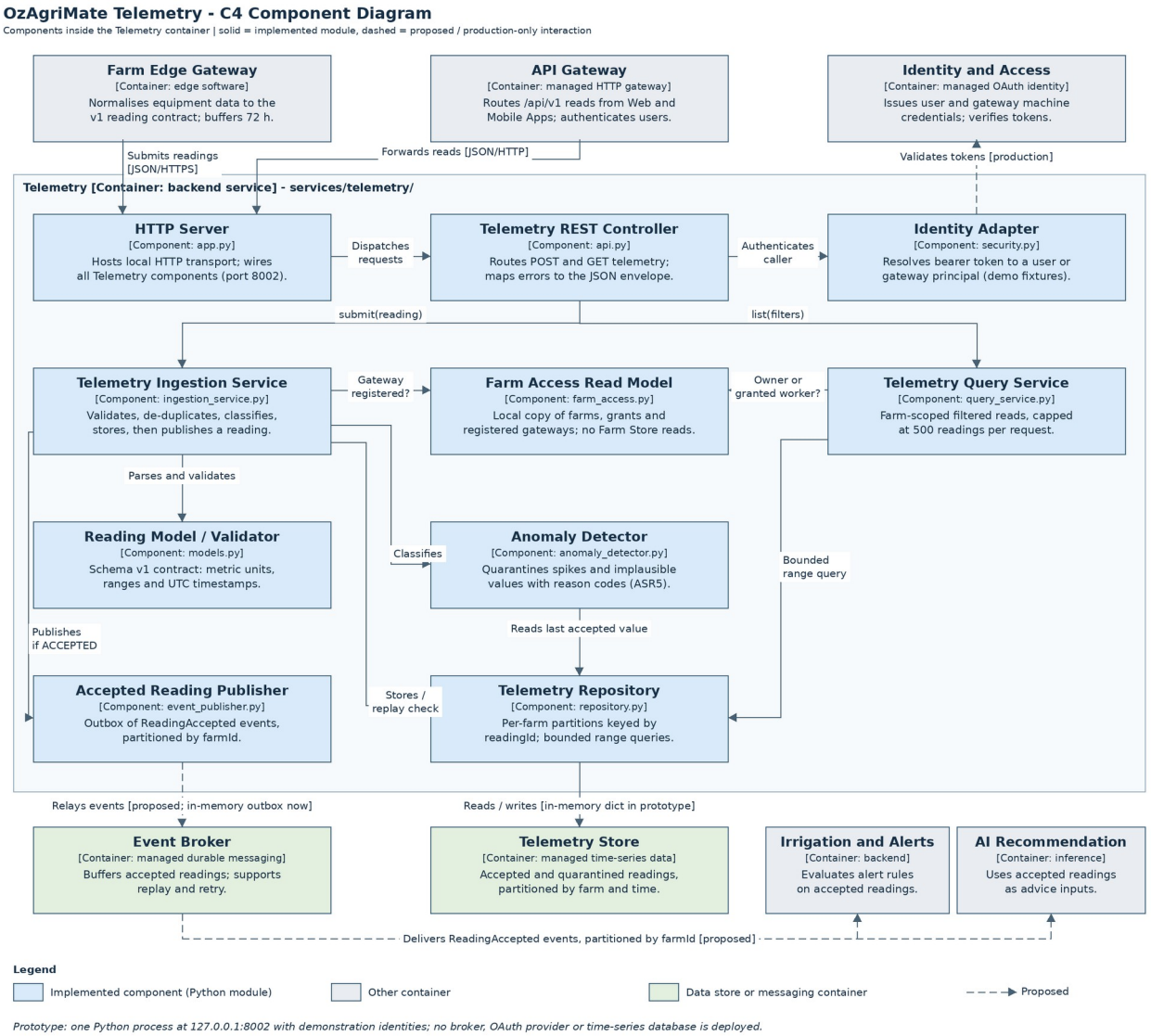
ADR 06 Edge-to-cloud reading contract

Field	Architectural decision record
Title	ADR-06 Adopt a versioned, idempotent JSON reading contract over HTTPS between Farm Edge Gateways and Telemetry
Status	Proposed
Context	Interoperability is a top driving QA: farms use LoRaWAN, NB-IoT, ISOBUS/ADAPT and vendor sensors over constrained, unreliable links (C2). ASR15 requires onboarding device types without changing ingestion; ASR16 requires versioned, backward-compatible APIs; gateways must resend buffered readings safely (ASR3, C3). Alternatives: accepting every vendor format in the cloud spreads protocol logic into Telemetry and changes it whenever hardware changes; devices publishing MQTT directly to a cloud broker need per-device credentials at scale and bypass central validation; binary formats such as Protocol Buffers save bandwidth but are harder for partners to adopt and test.
Decision	Gateways translate field protocols locally and POST one canonical JSON reading per request to /farms/{farmId}/telemetry over HTTPS with a farm-bound machine credential. Each reading carries schemaVersion, a stable gateway-assigned readingId, deviceId, paddockId, metric, value, canonical unit and UTC observedAt. Telemetry rejects unknown versions, extra fields and unit mismatches with 422, returns 201 for new readings, 200 for identical replays and 409 for conflicting reuse. ReadingAccepted events reuse these fields. New schema versions are added alongside the two previous versions (ASR16), never changed in place.
Consequences	Telemetry stays protocol-agnostic, and replays never duplicate readings or downstream actions. Gateways must maintain adapters and stable reading IDs. Per-reading JSON costs more bandwidth than binary batches; batching is a candidate for a later version. Supporting several schema versions adds testing and deprecation effort.
Compliance	Contract tests verify 201, 200, 409 and 422 behaviour for each schema version. Gateway certification checks unit mapping and readingId stability before field deployment.

Traceability: Interoperability (top-three driving QA); ASR1, ASR3, ASR5, ASR15, ASR16; constraints C2 and C3. Event fields satisfy the team integration contract (stable reading ID, farm ID, device ID, UTC time, metric, value, unit, schema version). Enables ADR-02 idempotent replay; complements ADR-04.

Telemetry component view

Figure 5 OzAgriMate Telemetry C4 Component Diagram



The view decomposes the Telemetry container into the modules implemented in services/telemetry/, so diagram and repository correspond one-to-one. Field-protocol adaptation happens on the Farm Edge Gateway (ADR-06), so Telemetry contains no vendor adapters. Interactions with the Event Broker and consuming services are dashed because no broker is deployed.

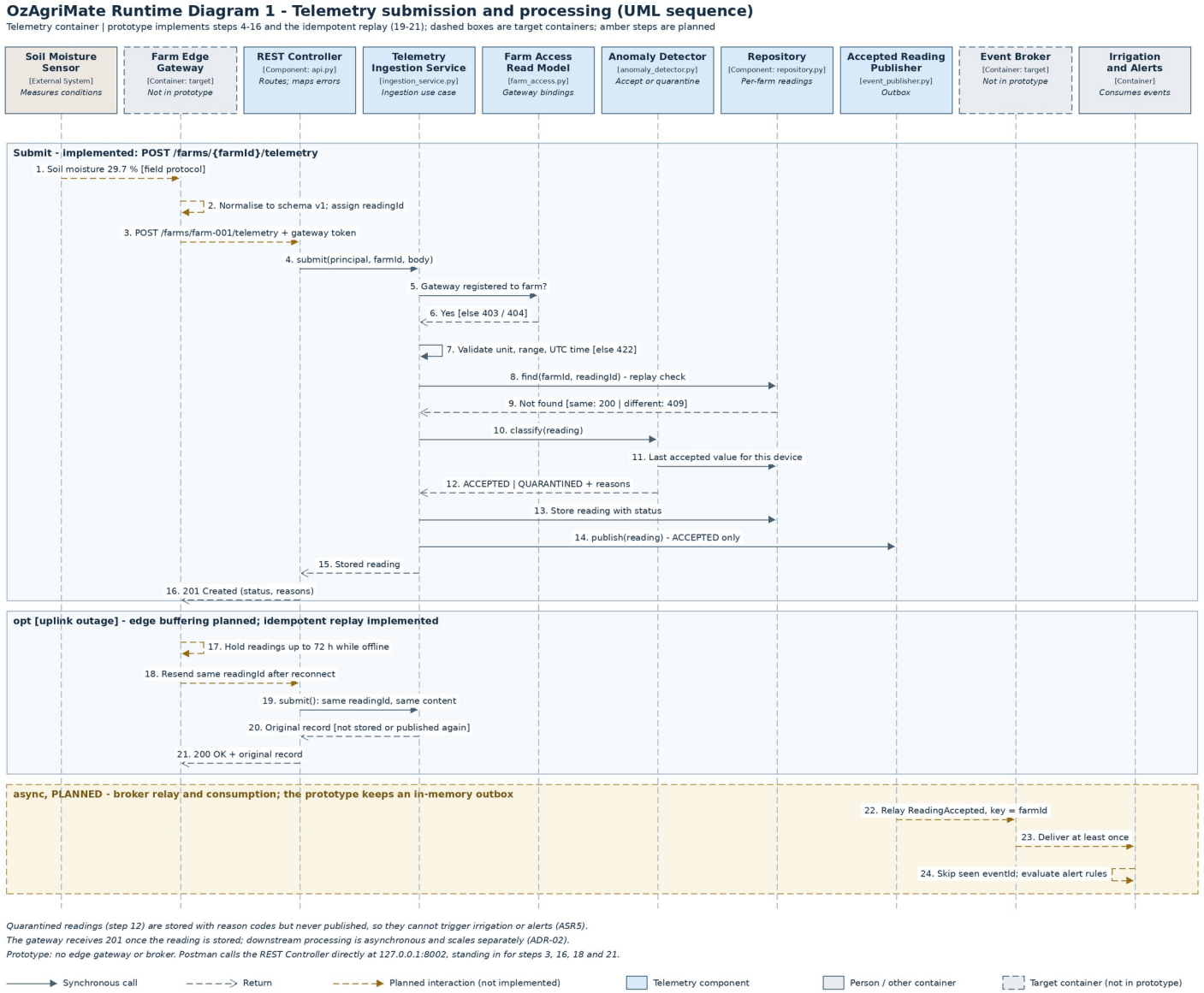
Component	Module	Responsibility
HTTP Server	app.py	Local HTTP transport and composition root, port 8002
Telemetry REST Controller	api.py	Routes POST and GET telemetry; maps errors to the JSON envelope
Identity Adapter	security.py	Resolves user or gateway principal (demonstration fixtures)
Telemetry Ingestion Service	ingestion_service.py	Authorise, validate, de-duplicate, classify, store, publish
Telemetry Query Service	query_service.py	Farm-scoped filtered reads capped at 500 rows
Farm Access Read Model	farm_access.py	Local copy of farms, grants and gateway bindings
Reading Model / Validator	models.py	Schema v1 contract, units, ranges and timestamps

Component	Module	Responsibility
Anomaly Detector	anomaly_detector.py	Spike and plausibility checks; quarantine reasons
Telemetry Repository	repository.py	Per-farm partitions keyed by readingId
Accepted Reading Publisher	event_publisher.py	Outbox of ReadingAccepted events, partitioned by farmId

Runtime view: telemetry submission and processing

Figure 6 OzAgriMate Runtime Diagram 1: telemetry submission and processing (UML sequence)

Interaction flow: Gateway submits normalised reading; Telemetry authorises, validates, de-duplicates and classifies it, stores it, then publishes accepted readings asynchronously.



Telemetry API design

Paths follow the team integration contract. Production versioning is proposed as /api/v1 at the gateway; the prototype exposes the unprefixed service paths at http://127.0.0.1:8002.

POST /farms/{farmId}/telemetry

Specification	Value
HTTP Method	POST
Description	Submit one sensor reading from a registered farm edge gateway
Endpoint URL	/farms/{farmId}/telemetry

Sample Request Body	{
	"schemaVersion": 1,
	"readingId": "sm-north-01-20261023T2045Z",
	"deviceId": "sm-north-01",
	"paddockId": "north-paddock",
	"metric": "soilMoisture",
	"value": 29.7,
	"unit": "%",
	"observedAt": "2026-10-23T20:45:00Z"
	}

Sample Response Body	{
	"readingId": "sm-north-01-20261023T2045Z",
	"farmId": "farm-001",
	"deviceId": "sm-north-01",
	"paddockId": "north-paddock",
	"metric": "soilMoisture",
	"value": 29.7,
	"unit": "%",
	"observedAt": "2026-10-23T20:45:00Z",
	"receivedAt": "2026-10-24T12:06:37Z",
	"status": "ACCEPTED",
	"quarantineReasons": [],
	"schemaVersion": 1
	}

Success status	201 Created (new reading); 200 OK (identical replay returns the original record)

GET /farms/{farmId}/telemetry

Specification	Value
HTTP Method	GET
Description	List a farm's sensor readings with filters and limits
Endpoint URL	/farms/{farmId}/telemetry
Sample Request Body	N/A
Sample Response Body	{ "farmId": "farm-001", "count": 2, "hasMore": false, "readings": [{ "readingId": "sm-north-01-20261023T2030Z", "farmId": "farm-001", "deviceId": "sm-north-01", "paddockId": "north-paddock", "metric": "soilMoisture", "value": 30.1, "unit": "%", "observedAt": "2026-10-23T20:30:00Z", "receivedAt": "2026-10-23T20:30:04Z", "status": "ACCEPTED", "quarantineReasons": [], "schemaVersion": 1

Specification	Value

	<pre> },
	  {
	   "readingId": "sm-north-01-20261023T2045Z",
	   "farmId": "farm-001",
	   "deviceId": "sm-north-01",
	   "paddockId": "north-paddock",
	   "metric": "soilMoisture",
	   "value": 29.7,
	   "unit": "%",
	   "observedAt": "2026-10-23T20:45:00Z",
	   "receivedAt": "2026-10-24T12:06:37Z",
	   "status": "ACCEPTED",
	   "quarantineReasons": [],
	   "schemaVersion": 1
	  }
	]
	}</pre>

Success status	200 OK

Sample response is for GET /farms/farm-001/telemetry?metric=soilMoisture&from=2026-10-23T20:30:00Z; all query parameters are optional.

receivedAt is assigned by the server, so its value differs on each run. A spike or implausible value is still stored (201) but with status QUARANTINED and reasons such as SPIKE_DETECTED or IMPLAUSIBLE_VALUE; it is never published as an accepted reading.

Validation and error behaviour

A POST body must contain exactly schemaVersion, readingId, deviceId, paddockId, metric, value, unit and observedAt. farmId comes from the URL and status from the service; caller-supplied farmId, status or receivedAt is rejected. schemaVersion must be 1. readingId is 1 to 80 letters, digits, '.', '_', ':' or '-'. deviceId and paddockId are lowercase slugs. Supported metrics and units: soilMoisture in % (0 to 100), soilTemperature in degC (-20 to 60), airTemperature in degC (-50 to 60) and rainfall in mm (0 to 500). observedAt is a UTC timestamp at most five minutes in the future.

GET accepts optional metric, deviceId, paddockId, from, to, status (ACCEPTED by default, QUARANTINED or ALL) and limit (1 to 500, default 100). Readings return in observation order; hasMore shows that the limit truncated the result. Unknown or repeated parameters are rejected.

Demonstration identities match the other services. demo-gateway-001 is registered to farm-001 and demo-gateway-002 to farm-002; only gateways may submit. demo-owner-a owns farm-001 and demo-worker-a holds a grant on it, so both may read it; demo-owner-b owns farm-002.

Status	Code	Relevant condition
400	INVALID_JSON	Malformed UTF-8 JSON or literal NaN/Infinity
401	UNAUTHENTICATED	Missing or invalid demonstration bearer identity
403	FORBIDDEN	Non-gateway submits a reading, or a gateway tries to read
404	FARM_NOT_FOUND	Farm absent, not accessible, or gateway not registered to it (identical bodies)
404	ROUTE_NOT_FOUND	Unknown path or malformed farm identifier
405	METHOD_NOT_ALLOWED	Unsupported method; Allow: GET, POST
409	READING_ID_CONFLICT	readingId reused with different content
413	BODY_TOO_LARGE	Body exceeds 65,536 bytes
415	UNSUPPORTED_MEDIA_TYPE	Content-Type is not application/json
422	VALIDATION_ERROR	Missing/extra fields, wrong unit, out-of-range value or invalid query parameter
422	UNSUPPORTED_SCHEMA_VERSION	schemaVersion is not 1

Error response example:

```
{"error":{"code":"READING_ID_CONFLICT","message":"readingId sm-north-01-20261023T2045Z was
already stored with different content."}}
```

Where the endpoints sit in the architecture

Both endpoints are served by the Telemetry container. In production, Farm Edge Gateways call POST over HTTPS with farm-bound machine credentials, and the Web and Mobile Apps call GET through the API Gateway. The REST Controller delegates to the Ingestion or Query Service, which authorises against Telemetry's own Farm Access Read Model rather than Farm Management's store and reads or writes the Telemetry Store through the Repository. Accepted readings leave the service only as ReadingAccepted events, consumed by Irrigation and Alerts and AI Recommendation, so Telemetry never waits for them.

Repository integration

As repository integrator, Aditya added a single start-up path and packaging for the whole team. The structure keeps one package per backend container from the Container Diagram, matching ADR-01.

Item	Purpose
run_all.py	Starts Farm Management (8001), Telemetry (8002) and Irrigation and Alerts (8003) in one terminal
requirements.txt	Records that the prototype needs only the Python 3.10+ standard library
README.md	Start-up, test, Postman and packaging instructions for all services
scripts/package_submission.py	Builds the clean code ZIP and re-runs all tests from the extracted copy
tests/test_run_all.py	Checks all three services start together and answer on their own ports

Validation evidence

Fourteen HTTP integration tests start a real local server with a fixed clock and cover the exact POST contract, idempotent replay, 409 conflicts, spike and rainfall quarantine (including that quarantined readings are not published), gateway-only submission, foreign-gateway denial, schema and unit validation, rejected server-owned fields, schema versions, ordered and filtered retrieval, windows and limits, worker access, cross-owner denial, invalid query parameters, authentication, media type and method handling.

Measure	Observed result
Automated tests (telemetry)	14 passed, 0 failed
Full repository suite	49 passed, 0 failed
Postman collection (Newman 6, fresh server)	postman/Aditya_Telemetry.postman_collection.json: 19 requests, 59 of 59 assertions passed
Combined run via run_all.py	Farm 33/33, Telemetry 59/59 and Irrigation 40/40 assertions passed
Clean ZIP check	Extracted archive passed the full test suite

The collection was executed with Newman, Postman's command-line runner, which executes the saved JavaScript assertions; the transcript is in docs/Aditya_Newman_Run.txt. Local results do not demonstrate production throughput, autoscaling, broker lag or 72-hour edge replay.

References

[1] Brown, S. (n.d.). Component diagram. C4 model. https://c4model.com/diagrams/component. Accessed 24 October 2026.

[2] Brown, S. (n.d.). Dynamic diagram. C4 model. https://c4model.com/diagrams/dynamic. Accessed 24 October 2026.

[3] Sharma, P., Singh, A., Wight, T., and Shears, B. (2026). OzAgriMate Software Architecture Proposal. Submitted ISYS3459 Assignment 2 report.

[4] Sharma, P. (2026). OzAgriMate Architecture and Farm API; Team Integration Contract. ozagrimate-a4 repository.

[5] RMIT University. (2026). ISYS3459 Assignment 4 Software Architecture Project activity instructions. Supplied course material.

[6] RMIT University. (2026). ADR Template. Supplied course material.